==========

==========

CHAPTER 18 — CACHE MAINTENANCE AND INTERACTION WITH THE MMU

==========

==========

A maximum-detail, RTOS/QNX-style technical breakdown of ARMv8 cache hierarchy, cache maintenance operations, coherency rules, DMA interactions, and how caches integrate with the MMU and TLB.

****

****

18.1 — Introduction to Cache Maintenance

****

****

ARMv8 systems use multi-level caches:

- **L1 I-cache** (instruction)
- **L1 D-cache** (data)
- **L2 unified cache**

- **L3 cache** (optional, multi-cluster)

Caches accelerate memory access but introduce complexity:

- stale data
- stale instructions
- stale translations
- DMA incoherence
- device memory hazards
- SMP ordering issues

Cache maintenance operations ensure correctness across:

- MMU
- TLB
- DMA
- devices
- SMP cores
- hypervisor transitions

This chapter explains **every cache maintenance instruction**, how it interacts with the MMU, and how to use it safely.

****

****

18.2 — ARMv8 Cache Maintenance Instructions

****

****

ARMv8 provides several cache maintenance operations:

Data Cache Maintenance

- `DC ZVA` — Zero cache line
- `DC IVAC` — Invalidate by VA to PoC
- `DC ISW` — Invalidate by set/way
- `DC CVAC` — Clean by VA to PoC
- `DC CSW` — Clean by set/way
- `DC CIVAC` — Clean+Invalidate by VA to PoC
- `DC CISW` — Clean+Invalidate by set/way

Instruction Cache Maintenance

- `IC IVAU` — Invalidate instruction cache by VA
- `IC IALLU` — Invalidate entire instruction cache
- `IC IALLUIS` — Invalidate instruction cache inner shareable

TLB Maintenance

- `TLBI <operation>` — invalidate TLB entries

Barriers

- `DSB SY`
- `DMB SY`
- `ISB`

These operations must be used **in precise sequences** to avoid corruption.

****

****

18.3 — Cache Coherency Domains

****

****

Cache coherency depends on shareability:

Inner Shareable

- L1/L2 coherent
- required for SMP correctness
- DMA buffers must be Inner Shareable

Outer Shareable

- L3 coherent
- multi-cluster systems

Non-Shareable

- no coherence
- used for device memory
- used for private scratch memory

****

****

18.4 — Cache Line States

****

****

Cache lines may be:

- **Clean** — matches memory
- **Dirty** — modified in cache, not in memory
- **Invalid** — not present or stale
- **Shared** — coherent across cores
- **Exclusive** — owned by one core

Cache maintenance operations manipulate these states.

****

****

18.5 — Clean Operations

****

****

DC CVAC (Clean by VA to PoC)

Writes dirty cache line to memory.

DC CSW (Clean by Set/Way)

Writes dirty cache line to memory using set/way indexing.

Use Cases

- before DMA reads
- before Stage-2 remapping
- before hypervisor migration
- before sharing memory with another core

Required Barrier

```
DSB  SY
```

****

****

18.6 — Invalidate Operations

****

****

DC IVAC (Invalidate by VA to PoC)

Invalidates cache line without writing back.

DC ISW (Invalidate by Set/Way)

Invalidates cache line using set/way indexing.

Use Cases

- after DMA writes
- after hypervisor remapping
- after page migration
- after modifying executable code

Required Barrier

```
DSB  SY
```

****

****

18.7 — Clean+Invalidate Operations

****

****

DC CIVAC (Clean+Invalidate by VA)

Writes back dirty line and invalidates it.

DC CISW (Clean+Invalidate by Set/Way)

Writes back dirty line and invalidates it using set/way indexing.

Use Cases

- full synchronization
- memory migration
- hypervisor page remapping
- safety-critical systems

Required Barrier

```
DSB  SY
```

****

****

18.8 — Zero Cache Line

****

****

DC ZVA

Zeroes a cache line efficiently.

Use Cases

- fast memset
- initializing buffers
- clearing memory before DMA

****

****

18.9 — Instruction Cache Maintenance

****

****

Instruction cache must be invalidated when:

- code is modified
- JIT compilers generate code
- hypervisor modifies guest code
- kernel loads modules
- user space loads shared libraries

IC IVAU

Invalidate instruction cache by VA.

IC IALLU

Invalidate entire instruction cache.

IC IALLUIS

Invalidate instruction cache inner shareable.

Required Sequence

```
DC CVAC, x0        // clean data cache
DSB SY             // ensure visibility
IC IVAU, x0        // invalidate instruction cache
DSB SY             // ensure completion
ISB                // flush pipeline
```

****

****

18.10 — Cache Maintenance and TLB

****

****

Cache maintenance interacts with TLB:

- page table updates require cache clean
- stale descriptors may remain in cache
- walk cache may contain stale entries
- TLB invalidation must follow cache maintenance

Correct Sequence

```
DC CVAC, descriptor
DSB SY
TLBI <operation>
DSB SY
ISB
```

****

****

18.11 — Cache Maintenance and Page Table Updates

****

****

Page table updates require:

1. Write descriptor
2. Clean descriptor cache line
3. DSB SY
4. TLBI
5. DSB SY
6. ISB

Why?

- descriptor must reach memory
- stale TLB entries must be removed
- pipeline must flush stale translations

Skipping any step causes **silent corruption**.

****

****

18.12 — Cache Maintenance and DMA

****

****

DMA engines operate outside CPU caches.

Before DMA Read

CPU must ensure memory is up-to-date:

```
DC CVAC, buffer
DSB SY
```

After DMA Write

CPU must invalidate stale cache lines:

```
DC CIVAC, buffer
DSB SY
```

DMA Buffers Must Be

- Normal memory
- Inner Shareable
- cacheable or non-cacheable depending on usage

MMIO Must Be

- Device-nGnRnE
- Non-Shareable
- non-cacheable

****

****

18.13 — Cache Maintenance and Device Memory

****

****

Device memory must:

- never be cached
- never be prefetched
- never be speculated
- never be cleaned or invalidated

Illegal

Running `DC CVAC` or `DC IVAC` on MMIO.

Required Ordering

```
STR x0, [MMIO]
DSB SY
```

****

****

18.14 — Cache Maintenance and SMP Systems

****

****

SMP systems require:

- Inner Shareable memory
- broadcast cache maintenance
- broadcast TLBI
- DSB SY for global visibility

Common SMP Bugs

- mapping RAM as Non-Shareable
- missing DSB after cache maintenance
- missing TLBI broadcast
- stale walk cache entries

****

****

18.15 — Cache Maintenance and Hypervisor

****

****

Hypervisor uses cache maintenance for:

- Stage-2 page table updates
- VM migration
- memory overcommit
- page swapping
- shadow page tables
- nested virtualization

Required Sequence

```
DC CVAC, descriptor
DSB SY
TLBI ALLE2
DSB SY
ISB
```

****

****

18.16 — Cache Maintenance and Nested Virtualization

****

****

Nested virtualization requires:

- nested cache maintenance
- nested TLBI
- nested BBM
- nested descriptor updates

Complexity

Nested cache maintenance must be carefully ordered to avoid:

- stale Stage-1 entries
- stale Stage-2 entries
- stale shadow entries

****

****

18.17 — Cache Maintenance and QNX/RTOS

****

****

QNX requires:

- deterministic cache maintenance
- strict ordering
- strict DMA rules
- strict device memory rules
- strict SMP rules

QNX uses:

- Inner Shareable for RAM
- Non-Shareable for MMIO
- deterministic barrier sequences

****

****

18.18 — Common Cache Maintenance Mistakes

****

****

Mistake 1 — Forgetting DSB SY

Cache operations may not complete.

Mistake 2 — Forgetting ISB

Pipeline may use stale instructions.

Mistake 3 — Cleaning MMIO

Causes device malfunction.

Mistake 4 — Forgetting TLBI

Stale TLB entries remain.

Mistake 5 — Incorrect DMA attributes

Causes data corruption.

****

****

18.19 — Full Cache Maintenance Example

****

****

Example: Updating page tables safely

```
STR x0, [descriptor]        // write descriptor
DC CVAC, descriptor         // clean cache line
DSB SY                      // ensure visibility
TLBI VAE1, x1               // invalidate TLB entry
DSB SY                      // ensure TLBI complete
ISB                         // flush pipeline
```

Result

- descriptor visible
- stale TLB entries removed
- pipeline updated
- ordering guaranteed

****

****

18.20 — Chapter Summary

****

****

This chapter covered:

- cache hierarchy
- cache maintenance instructions
- clean/invalidate operations
- instruction cache maintenance
- TLB interaction
- page table update rules
- DMA interaction
- device memory rules
- SMP rules
- hypervisor rules
- nested virtualization
- QNX/RTOS rules
- common mistakes
- full examples